2.1.11 Advanced Topic: Parts of Expressions

There are many times when you need to pick out or modify particular parts of an expression. The easiest way to do this, if your computer system allows it, is to select the parts graphically, using a pointing device such as a mouse.

If, however, your computer does not support this kind of interaction, or if you need to do these operations as part of a program, then you will need to read the rest of this section.

Section 1.2.4 described how you can pick out parts of lists by giving their integer "indices". The same approach can be used for arbitrary expressions, where now the indices label branches on the expression tree.

Picking out parts of arbitrary expressions is often much more complicated than picking out parts of lists. The problem is that the standard printed form of an expression may not correspond very closely to its tree representation. In setting up the printed form to mimic mathematical notation, the parts often get rearranged.

For small expressions, you can use `TreeForm[`*expr*`]` to see the explicit tree structure. For larger expressions, `FullForm[`*expr*`]` shows the arrangement of parts.

Here is an expression.

```
In[1]:= t = 1 + (3 + x)^2 / y
```

$$\text{Out[1]= } 1 + \frac{(3 + x)^2}{y}$$

This picks out the second term of `t`.

```
In[2]:= t[[2]]
```

$$\text{Out[2]= } \frac{(3 + x)^2}{y}$$

`FullForm` shows that the actual arrangement of parts in `t` is slightly different from what the usual printed form suggests.

```
In[3]:= FullForm[t]
Out[3]//FullForm=
  Plus[1, Times[Power[y, -1], Power[Plus[3, x], 2]]]
```

Knowing the actual arrangement of parts, it is straightforward to pick out a particular piece. This finds the first subpart of `t[[2]]`.

```
In[4]:= t[[2,1]]
```

$$\text{Out[4]= } \frac{1}{y}$$

This finds the part labelled by indices 2, 1, 1.

```
In[5]:= t[[2,1,1]]
Out[5]= y
```

This is what happens if you ask for a part that does not exist.

```
In[6]:= t[[7]]

Part::width: Part 7 of 1 + (3 + x)^2/y does not exist.

Out[6]= (1 + (3 + x)^2/y)[[7]]
```

You can think of the indices for a particular part in an expression as specifying the branches you need to take in order to get to that part by descending from the node of the tree.

You can reset parts of expressions in the same way as you reset parts of lists.

This resets a part of `t`.

```
In[7]:= t[[2,1,1]] = x
Out[7]= x
```

Now the form of `t` has been changed.

```
In[8]:= t

Out[8]= 1 + (3 + x)^2/x
```

You can find out the indices for parts with a particular form using the function `Position`.

This gives a list of the indices for the positions of the two `x`'s that appear in `t`.

```
In[9]:= Position[t, x]
Out[9]= {{2, 1, 1}, {2, 2, 1, 2}}
```

One way to apply a function to a particular part of an expression is to pick out that part, and then reset it to the result obtained by applying the function. An alternative is to use `MapAt`, which allows you to give the indices for a list of parts to which to apply a function.

This applies `f` to two parts of `t`.

```
In[10]:= MapAt[f, t, {{2, 1, 1}, {2, 2}}]

Out[10]= 1 + f[(3 + x)^2]/f[x]
```

You have to use a nested list even if you only want to apply `f` to a single part. (A single list `{2, 1, 1}` would be ambiguous.)

```
In[11]:= MapAt[f, t, {{2, 1, 1}}]

Out[11]= 1 + (3 + x)^2/f[x]
```

Section 1.2.4 discussed how you can use lists of indices to pick out several elements of a list at a time. You can use the same procedure to pick several parts in an expression at a time.

This picks out elements 2 and 4 in the list, and gives a list of these elements.

```
In[12]:= {a, b, c, d, e}[[{2, 4}]]
Out[12]= {b, d}
```

This picks out parts 2 and 4 of the sum, and gives a *sum* of these elements.

```
In[13]:= (a + b + c + d + e)[[{2, 4}]]
Out[13]= b + d
```

Any part in an expression can be viewed as being an argument of some function. When you pick out several parts by giving a list of indices, the parts are combined using the same function as in the expression.

expr`[[`*n*`]]`	the n^{th} part of *expr*
expr`[[-`*n*`]]`	the n^{th} part, counting from the end
expr`[[`n_1`, `n_2`, ...]]`	the part of *expr* labelled by indices from the tree
expr`[[`*n*`]] = `*value*	reset part of an expression
`MapAt[`*f*`, `*expr*`, {{`n_1`, `n_2`, ...}, {`m_1`, `m_2`, ...}, ...}]`	apply *f* to a list of parts of *expr*
`Position[`*expr*`, `*form*`]`	find the positions of parts matching *form* in *expr*
expr`[[{`n_1`, `n_2`, ...}]]`	give a combination of several parts of an expression

Functions for manipulating parts of expressions.